DS Assessment

August 31, 2026
Name: Aaditya Sogani
Reg No: 26MAG0064
Spectral Clustering on a Network Intrusion Flow Similarity Graph
CIC-IDS2017 (Benign-Monday + DoS-Wednesday combined)

---

1. Importing Required Libraries

```
import numpy as np
import pandas as pd
import networkx as nx
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.neighbors import kneighbors_graph
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, davies_bouldin_score
print("=" * 60)
print("SPECTRAL CLUSTERING ON A NETWORK FLOW SIMILARITY GRAPH")
print("=" * 60)

OUTPUT:
============================================================
SPECTRAL CLUSTERING ON A NETWORK FLOW SIMILARITY GRAPH
============================================================
```

2. Load and Prepare the Dataset

```
print("\n" + "=" * 60)
print("LOAD DATASET")
print("=" * 60)

OUTPUT:
============================================================
LOAD DATASET
============================================================
Sample shape: (40, 78)

Class counts in sample:
Label
Benign              10
DoS GoldenEye        8
DoS Hulk             8
DoS slowloris        8
DoS Slowhttptest     5
Heartbleed           1
Name: count, dtype: int64
```

3. Build the Flow Similarity Graph (k-Nearest-Neighbour Graph)

```
print("\n" + "=" * 60)
print("BUILD FLOW SIMILARITY GRAPH")
print("=" * 60)

OUTPUT:
============================================================
BUILD FLOW SIMILARITY GRAPH
============================================================

Number of nodes: 40
Number of edges: 152

Node degrees:
{0: 13, 1: 5, 2: 5, 3: 5, 4: 7, 5: 6, 6: 5, 7: 7, 8: 5, 9: 9, 10: 12, 11: 5, 12: 5, 13: 5,
14: 6, 15: 14, 16: 8, 17: 16, 18: 13, 19: 13, 20: 12, 21: 7, 22: 11, 23: 7, 24: 6, 25: 5,
26: 5, 27: 5, 28: 5, 29: 6, 30: 9, 31: 7, 32: 6, 33: 6, 34: 11, 35: 11, 36: 7, 37: 6, 38:
11, 39: 5}
```

4. Construct the Adjacency Matrix A

```
print("\n" + "=" * 60)
print("ADJACENCY MATRIX")
print("=" * 60)

OUTPUT:
============================================================
ADJACENCY MATRIX
============================================================

Adjacency Matrix Shape: (40, 40)

Adjacency Matrix (first 5 rows):
[[0. 1. 1. 0. 0. 1. 0. 0. 1. 0. 1. 0. 0. 0. 0. 0. 1. 1. 0. 0. 1. 1. 0. 0.
  0. 0. 0. 0. 0. 0. 0. 0. 1. 1. 1. 1. 0. 0. 0. 0.]
 [1. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0.
  0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 1. 0. 0. 0. 0. 0.]
 [1. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0.
  0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 1. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 1. 0. 0. 1.
  0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 1. 0. 1. 0. 0. 0. 0. 0. 1. 0. 0.
  0. 0. 0. 0. 0. 0. 1. 0. 0. 1. 0. 0. 0. 1. 0. 0.]]
```

5. Calculate the Degree Matrix D

```
print("\n" + "=" * 60)
print("DEGREE MATRIX")
print("=" * 60)

OUTPUT:
============================================================
DEGREE MATRIX
============================================================

Degree values:
[13.  5.  5.  5.  7.  6.  5.  7.  5.  9. 12.  5.  5.  5.  6. 14.  8. 16.
 13. 13. 12.  7. 11.  7.  6.  5.  5.  5.  5.  6.  9.  7.  6.  6. 11. 11.
  7.  6. 11.  5.]

Degree Matrix (first 5 rows):
[[13.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.
   0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.
   0.  0.  0.  0.]
 [ 0.  5.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.
   0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.
   0.  0.  0.  0.]
 [ 0.  0.  5.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.
   0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.
   0.  0.  0.  0.]
 [ 0.  0.  0.  5.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.
   0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.
   0.  0.  0.  0.]
 [ 0.  0.  0.  0.  7.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.
   0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.
   0.  0.  0.  0.]]
```

6. Construct the Graph Laplacian L = D - A

```
print("\n" + "=" * 60)
print("GRAPH LAPLACIAN")
print("=" * 60)

OUTPUT:
============================================================
GRAPH LAPLACIAN
============================================================

Laplacian Matrix Shape: (40, 40)

Laplacian Matrix (first 5 rows):
[[ 13.  -1.  -1.   0.   0.  -1.   0.   0.  -1.   0.  -1.   0.   0.   0.
    0.   0.  -1.  -1.   0.   0.  -1.  -1.   0.   0.   0.   0.   0.   0.
    0.   0.   0.   0.  -1.  -1.  -1.  -1.   0.   0.   0.   0.]
 [ -1.   5.  -1.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.
    0.   0.   0.   0.   0.   0.   0.  -1.   0.   0.   0.   0.   0.   0.
    0.   0.   0.   0.  -1.   0.  -1.   0.   0.   0.   0.   0.]
 [ -1.  -1.   5.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.   0.
    0.   0.   0.   0.   0.   0.   0.  -1.   0.   0.   0.   0.   0.   0.
    0.   0.   0.   0.  -1.   0.  -1.   0.   0.   0.   0.   0.]
 [  0.   0.   0.   5.   0.   0.   0.  -1.   0.   0.   0.   0.   0.   0.
    0.   0.   0.   0.   0.  -1.  -1.   0.   0.  -1.   0.   0.   0.   0.
    0.   0.   0.  -1.   0.   0.   0.   0.   0.   0.   0.   0.]
 [  0.   0.   0.   0.   7.   0.   0.   0.   0.  -1.   0.   0.   0.  -1.
    0.  -1.   0.   0.   0.   0.   0.  -1.   0.   0.   0.   0.   0.   0.
    0.   0.  -1.   0.   0.  -1.   0.   0.   0.  -1.   0.   0.]]
```

7. Eigenvalues and Eigenvectors of L

```
print("\n" + "=" * 60)
print("EIGENDECOMPOSITION OF LAPLACIAN")
print("=" * 60)

OUTPUT:
============================================================
EIGENDECOMPOSITION OF LAPLACIAN
============================================================

Smallest 10 eigenvalues:
[0.     0.5787 0.9141 2.2271 3.2151 3.3048 3.8583 4.4444 4.6988 4.7216]
```

8. Build Spectral Embeddings and Apply K-Means for k=2,3,4,5

```
print("\n" + "=" * 60)
print("SPECTRAL EMBEDDING AND K-MEANS CLUSTERING")
print("=" * 60)

OUTPUT:
============================================================
SPECTRAL EMBEDDING AND K-MEANS CLUSTERING
============================================================

-- k=2 ---
Cluster labels: [1 1 1 0 1 1 1 0 1 1 1 1 1 1 1 1 1 1 0 1 1 1 0 1 1 0 1 1 0 1 0 1 1 1 1 1
1 1 1]
Silhouette Score: 0.5989
Davies-Bouldin Index: 0.4257

-- k=3 ---
Cluster labels: [0 0 0 2 1 0 0 2 0 1 0 1 1 1 1 1 0 1 1 2 2 0 1 2 1 1 2 0 1 2 1 2 0 0 0 0 1
1 1 1]
Silhouette Score: 0.5487
Davies-Bouldin Index: 0.8579

-- k=4 ---
Cluster labels: [1 3 3 2 0 1 1 2 1 0 1 0 0 0 0 0 1 1 0 2 1 3 0 2 0 0 2 1 0 2 0 2 3 1 1 1 0
0 0 0]
Silhouette Score: 0.4840
Davies-Bouldin Index: 0.7885

-- k=5 ---
Cluster labels: [1 2 2 0 3 1 1 0 1 4 1 4 4 3 4 4 1 1 4 0 1 2 4 0 3 4 0 1 4 0 3 0 2 1 1 1 4
4 1 4]
Silhouette Score: 0.4752
Davies-Bouldin Index: 0.8735
```

9. Summarise Evaluation Metrics

```
print("\n" + "=" * 60)
print("CLUSTERING EVALUATION SUMMARY")
print("=" * 60)

OUTPUT:
============================================================
CLUSTERING EVALUATION SUMMARY
============================================================

 k  Silhouette Score  Davies-Bouldin Index
 2           0.5989                0.4257
 3           0.5487                0.8579
 4           0.4840                0.7885
 5           0.4752                0.8735
```

10. Select the Best Number of Clusters

```
print("\n" + "=" * 60)
print("BEST NUMBER OF CLUSTERS")
print("=" * 60)

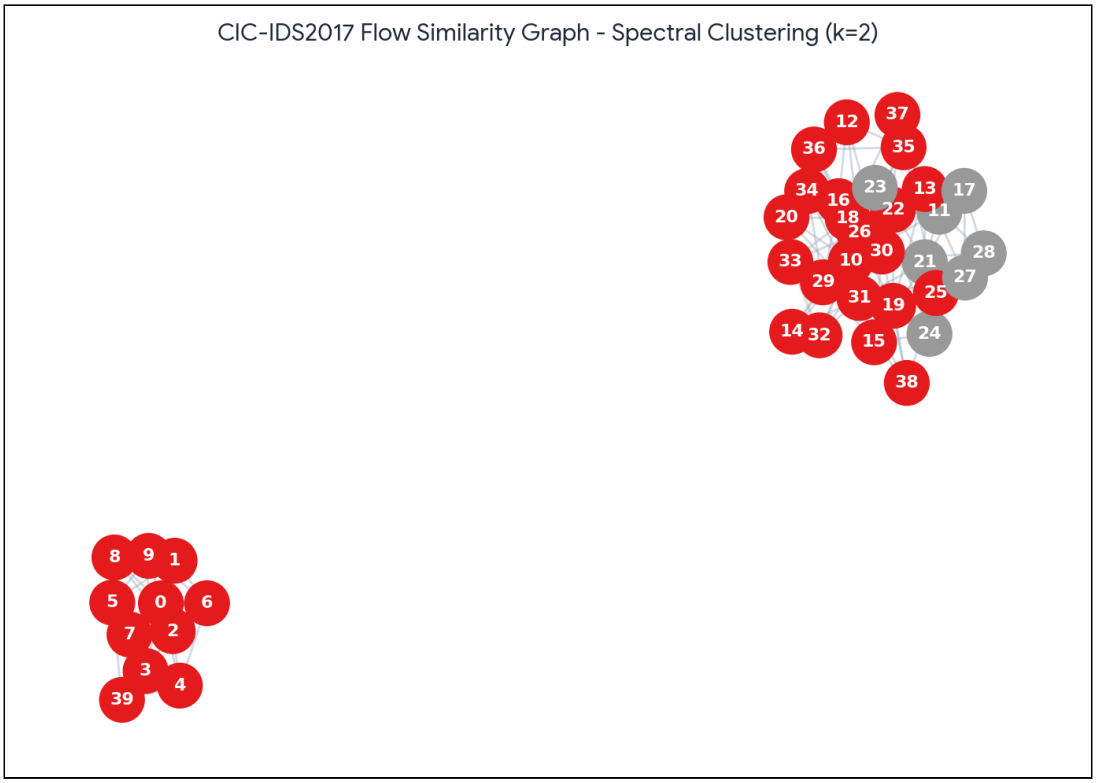
OUTPUT:
============================================================
BEST NUMBER OF CLUSTERS
============================================================

Best k by Silhouette Score (higher is better): 2 -> Silhouette = 0.5989
Best k by Davies-Bouldin Index (lower is better): 2 -> Davies-Bouldin = 0.4257
```

11. Visualise the Flow Similarity Graph with Detected Clusters

```
print("\n" + "=" * 60)
print("NETWORK VISUALISATION")
print("=" * 60)

OUTPUT:
============================================================
NETWORK VISUALISATION
============================================================
```



```
Visualisation saved for k=2 with 2 detected clusters.

Program completed successfully.
```